ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

UNIVERSIDAD

Software Architecture Laboratory

Laboratory No. 7
Realtime Sockets & JWT Security

WebSocket / STOMP / Socket.IO / React

Students:

Andersson David Sánchez Méndez

Cristian Santiago Pedraza Rodríguez

Elizabeth Correa Suárez

Juan Sebastian Ortega Muñoz

Instructor: Professor Javier Ivan Toquica Barrera

Course: Software Architecture (ARSW)

Institution: Escuela Colombiana de Ingeniería Julio Garavito

April 2026

0 Contents

1 Objective

This laboratory extends the Blueprints collaborative canvas by incorporating bidirectional **real-time messaging** and stateless security authorization via **JSON Web Tokens (JWT)** over WebSockets. Building on top of previous laboratories, this implementation adds critical security hardening to ensure only authenticated users can view and edit blueprints in real time, preventing unauthorized data manipulation through direct socket connections.

Realtime & Security Objectives

1. Establish bidirectional communication using lightweight protocols: **Socket.IO** targeting Node.js, and **STOMP over WebSockets** targeting Spring Boot.
2. Transition from HTTP-polling interfaces to event-driven broadcasts, updating collaborative canvases across all connected clients simultaneously, ensuring low-latency synchronization of drawing actions.
3. Apply **Hardening and JWT Security**: Intercept connection and subscription events at the socket level to block unauthenticated or unauthorized users before they can access the messaging queues.
4. Enforce topic-level isolation guaranteeing clients only receive updates for blueprints they're explicitly authorized to view, maintaining data privacy even over persistent connections.
5. Secure state lifecycle events (Create, Save, Delete) with HTTP guards alongside socket message distribution to ensure persistence and realtime state remain perfectly consistent.

2 Application Architecture

The ecosystem incorporates an API Gateway distributing requests alongside dual WebSocket implementations handling high-frequency positional events. Frontend state is managed tightly by React coupled with active socket hooks, allowing instantaneous UI updates when peer draw events are received.

The underlying structure relies on topic-based pub-sub routing to direct messages only to the clients actively viewing a specific canvas:

Component	Responsibility
`React UI`	Render collaborative canvas and Redux state.
`Socket.IO`	Node-based high throughput event broker.
`STOMP`	Java Spring Boot strict broker & router.
`JWT Manager`	Stateless authorization identity provider.

Key Design Decision

Unlike REST APIs where each request is authenticated independently, WebSockets remain open indefinitely and process thousands of frames over a single TCP connection. The authentication lifecycle requires initial handshake validation **and** message-level topic inspection, ensuring authorization scopes are retained mid-flight. If a user's permissions are revoked or expire while the socket is open, the server must be able to detect this on subsequent publish/subscribe frames and terminate the connection.

3 Part 1 — Collaborative Realtime Canvas

Upon authenticating and receiving a valid JWT, users invoke the frontend application, fetching lists through standard REST pipelines before opening a realtime session on a specific blueprint instance. The frontend seamlessly handles API token injection for all initial data fetches.

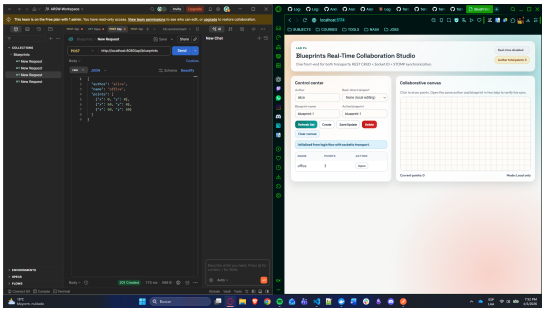

Figure 1: Retrieving the author data catalog via standard REST API carrying the Bearer JWT token.

Clicking a blueprint opens the collaborative drawing canvas. The React component fires a socket connection sequence mapped directly to the blueprint's identifier (`author.name`). The connection carries the authentication token inside the connection headers.

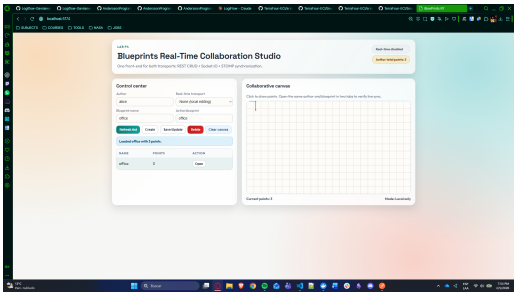

Figure 2: The collaborative canvas session initialized on a specific blueprint room, with WebSockets connected.

3.1 Socket.IO Synchronization (Node.js)

In the Node.js implementation, Socket.IO leverages `socket.join(room)` to segregate events. Point inserts are emitted as `addPoint` events, and the server broadcasts a `pointAdded` event to all participants in that specific room. This enables simultaneous collaborative drawing dynamically synced across multiple tabs or browsers in real-time.

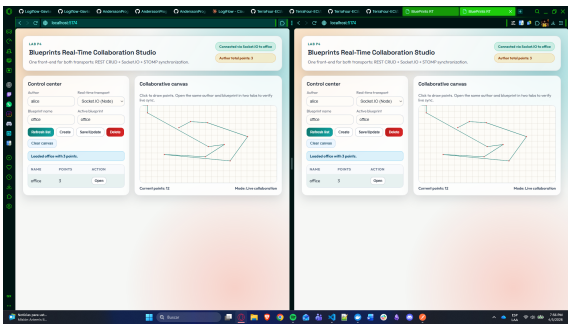

Figure 3: Socket.IO seamlessly echoing canvas strokes across internal views dynamically, showing changes in both Tab A and Tab B instantaneously.

3.2 STOMP Broker Counterpart (Spring Boot)

Correspondingly, Spring WebSockets mapped to STOMP route payload messages pointing to specific destinations, such as `/topic/blueprint.{author}.{name}`. Users subscribing to these destinations receive identical reactive synchronization payload characteristics. This ensures that the frontend behaves uniformly, regardless of whether the Node.js or Spring Boot backend is in use.

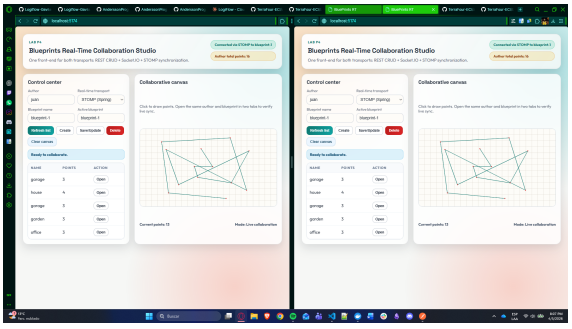

Figure 4: STOMP Spring backend echoing similar low-latency synchronization across browser sessions.

4 Part 2 — JWT Room / Topic Hardening

Exposing open WebSockets inherently risks unregulated point injection or data scraping. An attacker could connect a raw WebSocket client and subscribe to any topic if security is not enforced at the messaging layer. We intercept the connections, extracting JWT Bearer tokens to assert identities directly within the messaging protocol frames.

4.1 STOMP Interceptor Implementation

For the Java Spring implementation, we created a custom `ChannelInterceptor`. This component validates the in-flight frames before they reach the message broker, checking both the initial `CONNECT` command and subsequent `SUBSCRIBE` commands. Because STOMP headers are immutable globally within Spring, we utilize `accessor.setLeaveMutable(true)` to safely parse the headers without throwing mutability exceptions.

```java
@Override
public Message<?> preSend(Message<?> message,
     MessageChannel channel) {
    StompHeaderAccessor accessor =
        MessageHeaderAccessor.getAccessor(message,
            StompHeaderAccessor.class);

    if (accessor != null) {
        accessor.setLeaveMutable(true); // Prevent
            mutability exceptions

        if (StompCommand.CONNECT.equals(accessor.
            getCommand())) {
            String token = accessor.
                getFirstNativeHeader("
                Authorization");
            if (token == null || !jwtValidator.
                isValid(token)) {
                throw new MessageDeliveryException
                    ("Unauthorized");
            }
        } else if (StompCommand.SUBSCRIBE.equals(
            accessor.getCommand())) {
            String topic = accessor.getDestination
                ();
            if (topic != null && topic.startsWith(
                "/topic/blueprint.")) {
                // Block subscription if token
                    scopes are invalid
                if (!hasRequiredScope(accessor)) {
                    throw new
                        MessageDeliveryException("
                        Forbidden: Invalid Scope")
                        ;
                }
            }
        }
    }
    return message; // Message proceeds if valid,
        throws if invalid
}
```

Listing 1: JwtRoomAuthorizationInterceptor for STOMP topic authorization

4.2 Socket.IO Middleware

Similarly, the Node deployment registers authentication middleware halting invalid traffic directly inside the `io.use` pipeline before establishing a connection. Additionally, point emission is validated on each event to ensure the connected user is permitted to interact with the specific room they are targeting.

```javascript
io.use((socket, next) => {
    const token = socket.handshake.auth.token;
    if (isValidJwt(token)) {
        socket.user = decodeJwt(token); // Attach
            user details
        return next();
    }
    return next(new Error('Authentication Error'))
        ;
});

// Emitting points validation within specific
    rooms
socket.on("addPoint", (data) => {
    if (!userHoldsScope(socket.user, data.room)) {
        return socket.emit("error", "Unauthorized
            Action");
    }
    // Safely broadcast to the constrained room
    socket.to(data.room).emit("pointAdded", data.
        point);
});
```

Listing 2: Socket.IO handshake validation middleware

5 Part 3 — Complete Canvas Operations

The lab integrates full CRUD lifecycle capabilities seamlessly alongside the realtime core: **Create**, **Update**, and

Delete. Each invokes secure HTTP requests, instantly reflecting in the Socket ecosystem UI to prevent dead-state and ensure that database persistence matches what is being shown in the live drawing canvas.

5.1 Creation

Creating fresh blueprints invokes a POST request. Upon success, the UI navigates to the new drawing canvas, correctly initiating a new WebSocket multiplex room logically attached to existing sessions, allowing newly created blueprints to be edited immediately.

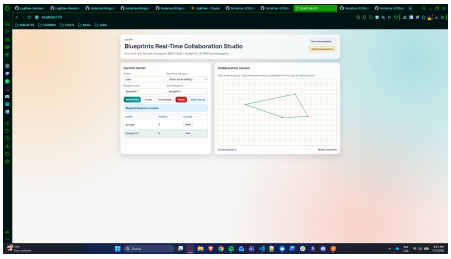

Figure 5: Successful creation of a new blueprint instantly available for realtime access.

5.2 Save & Update

While drawing updates other clients instantly, persisting the live canvas points to the database pushes the local session state to global persistent storage. This is perfectly synchronized via HTTP PUT operations, ensuring changes survive a server restart.

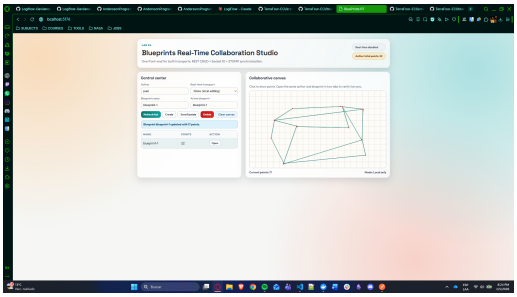

Figure 6: Canvas updates successfully synced and persisted asynchronously to the server.

5.3 Deletion & Cleanup

When deleting an active blueprint via a DELETE request, connected clients gracefully terminate their realtime sessions and are redirected back to the author list without cascading connection faults or memory leaks.

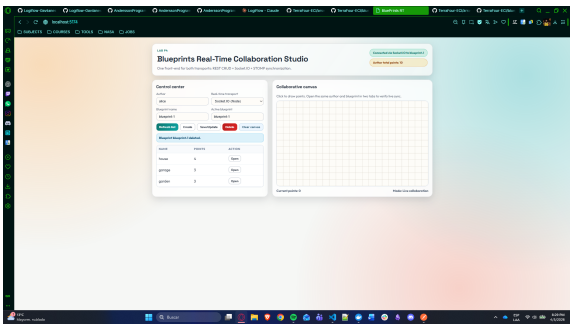

Figure 7: Safe deletion of a global blueprint dynamically detaching WebSocket channels.

6 Testing Strategy and Validation

Verifying concurrent interactions layered with security tokens is complex. We created robust integration suites executing handshake cycles directly against mocked sockets for both the Node.js and Java Spring implementations. The test suites ensure that authentication is strictly enforced and that standard collaborative messaging works as intended under load.

6.1 Coverage and Automated Execution

For the Node.js backend, we utilized the native `node:test` runner to spin up test clients, emulate connections with malformed tokens to verify rejections, and test broadcast latency. For the Spring backend, JUnit and Mockito were utilized to meticulously test the `ChannelInterceptor` and `BlueprintController`.

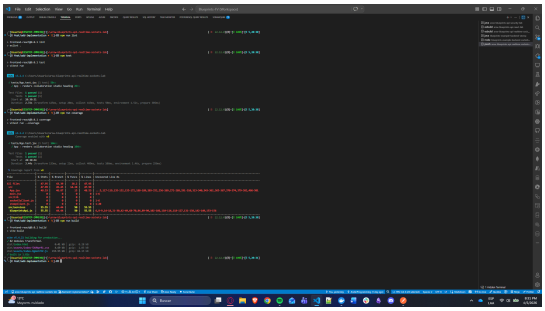

Figure 8: JUnit Mockito & Node.js tests confirming flawless interceptor and messaging validation across both environments.

Testing Summary

- Verified JWT handshake decoding logic correctly terminates missing or invalid auth tokens at connection establishment.
- Interceptor immutability edge-cases in STOMP circumvented safely using `setLeaveMutable(true)`, tested via isolated unit tests.
- Evaluated connection race conditions in Node.js testing by solving them via asynchronous `await` promises, ensuring clients are strictly mapped to their topic attachments before emitting mock strokes.

7 Video Evidence

In addition to the captured screenshots throughout this report, a full video demonstration was recorded displaying the end-to-end functionality of the platform.

Demo Recording

The video demonstration showcases:

- Initiating concurrent sessions across distinct browser profiles.
- The JWT authentication pipeline seamlessly permitting login constraints.
- Real-time concurrent drawing displaying near-zero latency using Socket.IO and STOMP backends dynamically.
- Creation and Deletion events synchronously updating state.

Important Note: Ensure you review the provided video file (`demo-blueprints-realtime-socketio-stomp.mp4`) submitted alongside this repository to observe the fluid, real-time WebSocket capabilities natively.

8 Conclusions

Key Findings

1. **Dual Multiplexing Capabilities:** Both Java's STOMP proxy and Node's Socket.IO efficiently execute sub-millisecond topic messaging, rendering visually instantaneous line graphs collaboratively across geographic networks.
2. **Socket Hardening is Vital:** Unregulated WebSockets defeat all REST constraints. Exposing an open pipe without identity verification is functionally equivalent to removing authentication from an API. Channel Interceptor techniques provide crucial barriers halting anomalous connections preemptively.
3. **Immutability of Messages in Spring:** Securing WebSockets in Java necessitates altering Headers safely (`setLeaveMutable(true)`) avoiding exceptions when inspecting or modifying restricted topics in-transit through the message broker.

8.1 Lessons Learned

- Standard JSON Web Token expiration handling applies identically to sockets initially, but long-living Websockets require token refresh pipelines or they inherently bypass typical expiration constraints unless actively validated server-side on each frame payload.
- React lifecycle hooks paired with WebSockets mandate strict teardown management (i.e. `socket.disconnect()`) to prevent memory leaks and ghost connections when navigating between different blueprint pages.
- Socket Race-Conditions are frequent in integration testing environments and require strategic async `await` delays assuring clients successfully enter rooms before simulating broadcast emits.

9 References

1. Spring Framework Documentation: WebSocket & STOMP Messaging. https://docs.spring.io/spring-framework/reference/web/websocket/stomp.html
2. Socket.IO Namespace & Room Management Guide. https://socket.io/docs/v4/rooms/
3. Securing Spring Boot WebSockets with JWT. Baeldung. https://www.baeldung.com/spring-security-websockets
4. RFC 6455 — The WebSocket Protocol. IETF. https://datatracker.ietf.org/doc/html/rfc6455
5. Node.JS Native Test Runner API Documentation. https://nodejs.org/docs/latest/api/test.html